

Lambda

Lambda – Motivation

Früher, ohne Lambda, war alles schlimmer! Wenn man „mal eben“ eine Collection umsortieren wollte, war das ein ziemlicher Akt! 🤖

```
void main() {  
  
    var persons = new LinkedList<Person>(List.of(  
        new Person("Mark", "Zuckerberg"),  
        new Person("Jeff", "Bezos"),  
        new Person("Bill", "Gates"),  
        new Person("Tim", "Cook")));  
  
    persons.sort(new SortByFirstnameComparator());  
    IO.println(persons);  
  
    persons.sort(new SortByLastnameComparator());  
    IO.println(persons);  
}
```

```
record Person(String firstname, String lastname) {  
    public String toString() {  
        return firstname + " " + lastname;  
    }  
}
```

```
import java.util.Comparator;  
  
class SortByFirstnameComparator implements Comparator<Person> {  
    @Override  
    public int compare(Person p1, Person p2) {  
        return p1.firstname().compareTo(p2.firstname());  
    }  
}
```

Verwende die vorhandene compareTo-Methode in der Klasse String.

```
class SortByLastnameComparator implements Comparator<Person> {  
    @Override  
    public int compare(Person p1, Person p2) {  
        return p2.lastname().compareTo(p2.lastname());  
    }  
}
```

🤖 Für jede neue Art der Sortierung musste eine neue Klasse erstellt und das Comparator<T>-Interface implementiert werden!

Ausgabe:

```
[Bill Gates, Jeff Bezos, Mark Zuckerberg, Tim Cook]  
[Jeff Bezos, Tim Cook, Bill Gates, Mark Zuckerberg]
```

Anonyme Klassen

Durch **anonyme Klassen** musste man nicht länger jedes Mal eine neue Klasse erstellen, um ein Interface zu implementieren. Aber schön ist das trotzdem noch nicht... 🙄

```
record Person(String firstname, String lastname) {  
    public String toString() {  
        return firstname + " " + lastname;  
    }  
}
```

```
void main() {  
  
    var persons = new LinkedList<Person>(List.of(  
        new Person("Mark", "Zuckerberg"),  
        new Person("Jeff", "Bezos"),  
        new Person("Bill", "Gates"),  
        new Person("Tim", "Cook")));  
  
    persons.sort(new Comparator<Person>() {  
        @Override  
        public int compare(Person p1, Person p2) {  
            return p1.firstname().compareTo(p2.firstname());  
        }  
    });  
    IO.println(persons);  
  
    persons.sort(new Comparator<Person>() {  
        @Override  
        public int compare(Person p1, Person p2) {  
            return p1.lastname().compareTo(p2.lastname());  
        }  
    });  
    IO.println(persons);  
}
```

Hier wird jeweils eine neue **anonyme Klasse** **temporär** erzeugt, die das `Comparator<T>`-Interface implementiert.

Ausgabe:

```
[Bill Gates, Jeff Bezos, Mark Zuckerberg, Tim Cook]  
[Jeff Bezos, Tim Cook, Bill Gates, Mark Zuckerberg]
```

Lambda to the rescue!

Mit **Lambdas** kann man „mal eben“ eine Implementation für ein Interface erzeugen und als Parameter übergeben. 😊

```
void main() {  
    var persons = new LinkedList<Person>(List.of(  
        new Person("Mark", "Zuckerberg"),  
        new Person("Jeff", "Bezos"),  
        new Person("Bill", "Gates"),  
        new Person("Tim", "Cook"))));  
  
    persons.sort((p1, p2) -> p1.firstname().compareTo(p2.firstname()));  
    IO.println(persons);  
  
    persons.sort((p1, p2) -> p1.lastname().compareTo(p2.lastname()));  
    IO.println(persons);  
}
```

```
record Person(String firstname, String lastname) {  
    public String toString() {  
        return firstname + " " + lastname;  
    }  
}
```

Ausgabe:

```
[Bill Gates, Jeff Bezos, Mark Zuckerberg, Tim Cook]  
[Jeff Bezos, Tim Cook, Bill Gates, Mark Zuckerberg]
```

Lambda – Definition



- Ein Lambda-Ausdruck ist ein (kurzer) Codeblock, der
 - optional Parameter entgegennimmt und
 - optional einen Wert zurückgibt.
 - **Parameter und Rückgabewert** werden durch ein **funktionales Interface** festgelegt.
 - **Jedes Lambda basiert auf einem funktionalen Interface.**
- Lambda-Ausdrücke sind ähnlich wie Methoden, sie
 - benötigen **keine Namen** (Implementation ist anonym, Methodename kommt vom Interface),
 - werden **nicht sofort ausgeführt** (sind lazy),
 - sind (möglichst) **zustandslos** (pure),
 - können direkt **innerhalb einer anderen Methode implementiert** werden.

Functional Interfaces

Lambdas basieren in Java auf [Functional Interfaces](#), das sind Interfaces mit **genau einer abstrakten** (genau eine nicht implementierten) Methode.

```
@FunctionalInterface
public interface MeaningOfLife {
    int calculate();
}
```

Anonyme Klasse statt Lambda

Bevor es Lambdas in Java gab, konnten Interfaces mithilfe von anonymen Klassen implementiert werden.

```
@FunctionalInterface
public interface MeaningOfLife {
    int calculate();
}

void main() {
    // Das Interface als anonyme Klasse implementiert.
    MeaningOfLife meaningOfLife = new MeaningOfLife() {
        @Override
        public int calculate() {
            return 42;
        }
    };

    // Führe die Methode in der anonymen Klasse aus.
    IO.println(meaningOfLife.calculate());
}
```

Ausgabe:
42

Lambda mit Rückgabewert

Lambdas sind deutlich kürzer als anonyme Klassen. Die Lambda-Implementierung wird durch einem Pfeil -> eingeleitet. Wenn es keine Parameter gibt, muss eine leere Klammer () vor dem -> stehen.

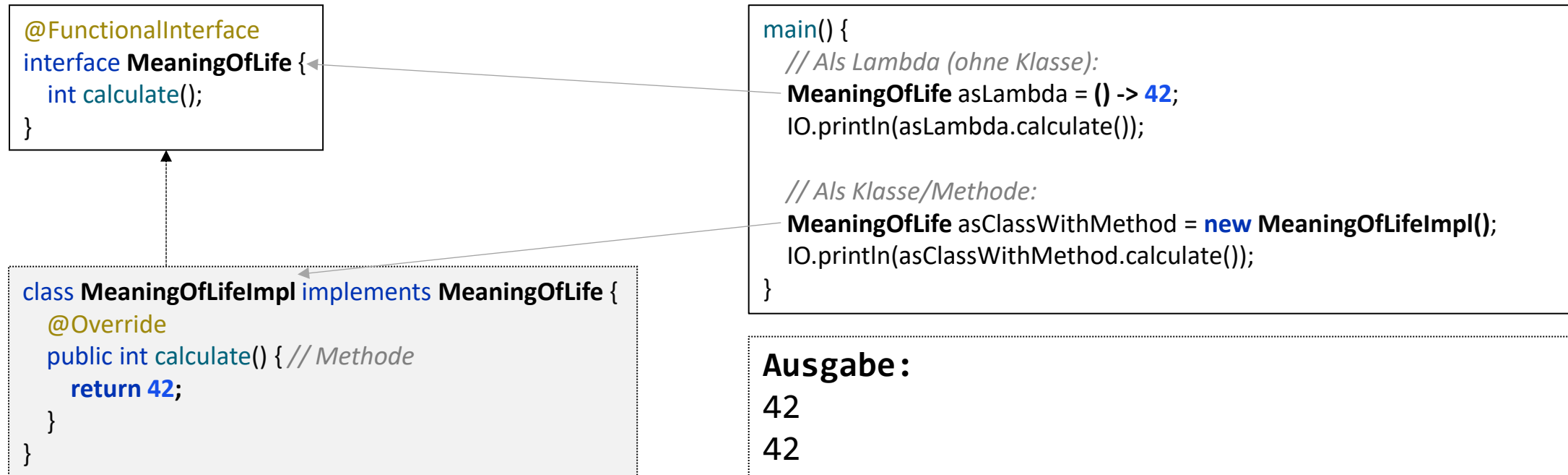
```
@FunctionalInterface
public interface MeaningOfLife {
    int calculate();
}

void main() {
    // Das Interface als Lambda implementiert.
    MeaningOfLife meaningOfLife = () -> 42;

    // Führe das Lambda aus.
    IO.println(meaningOfLife.calculate());
}
```

Ausgabe:
42

Implementation: Lambda vs Methode



Als Methode würde man eine Klasse erzeugen, um das gewünschte Interface zu implementieren.

Als Lambda kann man sich den Schritt (eine Klasse erzeugen) sparen.

Lambda mit Parameter

Parameter werden bei Lambdas in den Runden Klammern angegeben. Java kann die Datentypen für die Lambda-Parameter automatisch ermitteln, daher kann man die Datentypen auch weglassen. 🤖

```
@FunctionalInterface
public interface MoneyPrinter {
    void amount(int value);
}

void main() {
    // Definiere, was das Lambda mit dem Parameter tun soll.
    MoneyPrinter euroPrinter = (int value) -> IO.print("€" + value);
    MoneyPrinter dollarPrinter = (int value) -> IO.print("$" + value);

    // Führe das Lambda aus.
    euroPrinter.amount(42);
    dollarPrinter.amount(42);
}
```

Falls man den Datentyp des Parameters mit angibt, muss der Parameter inklusive Parametertyp in runden Klammern geschrieben werden.

Ausgabe:
€42
\$42

Lambda – Schreibweisen

Es gibt verschiedene Schreibweisen für Lambdas: mit/ohne runde/geschweifte Klammern.

Variante 1: *parameter* -> *expression* oder { *code block*; }

Beispiel: `persons.forEach(person -> IO.println(person.name));`

Beispiel: `persons.forEach(person -> { IO.println(person.name); });`

Lambda – Schreibweisen

Es gibt verschiedene Schreibweisen für Lambdas: mit/ohne runde/geschweifte Klammern.

Variante 2: *(parameter) -> expression oder { code block; }*

Beispiel: `persons.forEach((person) -> IO.println(person.name));` // Ohne { } sind nur "One-Liner" erlaubt.

Beispiel: `persons.forEach((person) -> {
 IO.println(person.name);
 // Innerhalb von { } sind beliebig viele weitere Zeilen erlaubt.
});`

Beispiel: `persons.forEach((Person person) -> IO.println(person.name));`

Der Parametertyp wird in der Regel weggelassen.

Die Schreibweise mit Parametertyp erfordert runde Klammern.

Lambda – Schreibweisen

Klammern sind nötig, wenn mehr als ein Parameter oder Expression verwendet werden.

Ab zwei Parametern sind die runden Klammern () Pflicht.

Variante 3: ***(parameter1, parameter2) -> expression oder { code block; }***

Beispiel: `persons.stream().sorted((p1, p2) -> p1.name.compareTo(p2.name));`

Beispiel:
`persons.stream().sorted((p1, p2) -> {
 return p1.name.compareTo(p2.name);
});`

Bei der Kurzschreibweise ohne { } muss man das return weglassen.

Lambda – Schreibweisen

Es gibt eine weitere Schreibweise für Lambdas mit [Methodenreferenz](#).

Variante 4: ***Klassennamen::methodName(parameter)***

Lambda: `numbers.forEach(number -> IO.println(number));`

Methodenreferenz: `numbers.forEach(IO::println);`

Lambda: `List.of("a", "b", "c").forEach(letter -> letter.toUpperCase());`

Methodenreferenz: `List.of("a", "b", "c").forEach(String::toUpperCase);`

Lambda: `numbers.forEach(number -> new Person(number));`

Methodenreferenz: `numbers.forEach(Person::new)`

Lambda: `numbers.stream().sorted((n1, n2) -> n1.compareTo(n2));`

Methodenreferenz: `numbers.stream().sorted(Integer::compareTo);`

Lambda – Methodenreferenz Einschränkungen 1/2

Methodenreferenzen sind oft kürzer als die Lambda-Schreibweise, allerdings sind Methodenreferenzen nicht möglich, sobald auf einem Parameter zugegriffen wird.

```
void main() {
```

```
    // 1. Kein Zugriff auf den Parameter.
```

```
    List.of(new Student("Elona")).forEach(student -> IO.println(student));
```

```
    List.of(new Student("Elona")).forEach(IO::println); // Äquivalent mit dem obigen Lambda.
```

```
    // 2. Zugriff auf dem Parameter. Schreibweise als Methodenreferenz nicht möglich.
```

```
    List.of(new Student("Elona")).forEach(student -> IO.println(student.name()));
```

```
    // Keine Methodenreferenz-Schreibweise zum obigen Lambda möglich.
```

```
}
```

Lambda – Methodenreferenz Einschränkungen 2/2

Methodenreferenzen sind nur auf implementierte Methoden möglich, aber nicht auf abstrakte Methoden ohne Implementierung.

@FunctionalInterface

interface MeaningOfLife {

int nothing(); *// Methode ohne Implementierung.*

default int calculate() { *// Methode mit Implementierung.*

return 42;

}

void main() {

// 1. Methodenreferenz funktioniert nicht mit abstrakten (nicht-implementierten) Methoden.

// MeaningOfLife lambda = MeaningOfLife::nothing; // nicht kompilierbar

// 2. Methodenreferenz funktioniert nur mit implementierten Methoden.

MeaningOfLife lambda = MeaningOfLife::calculate;

IO.println(lambda.calculate());

}

}

Pure Lambda



Lambdas sollen "pure" sein, d. h. **keine Seiteneffekte bzw. keinen Zustand** haben, damit sich Higher-Order Functions **vorhersagbar (deterministisch)** und **parallelisierbar** ausführen lassen:

1. Pure Lambda nimmt keine nach außen sichtbaren Veränderungen vor.
2. Pure Lambda hängt nicht von etwas ab, das sich von außen verändern kann.

Pure Lambda: Beispiel 1/2

Vermeide Lambdas, die etwas außerhalb manipulieren!
(wichtig im Bezug auf Multi-Threading sowie Wartbarkeit)



```
void main() {  
    var result = new ArrayList<Integer>();  
    Stream.of(1, 2, 3)  
        .map(number -> result.add(number * 2)) // Not pure!  
        .toList();                             // Änderung nach außen!!  
    IO.println(result);  
}
```

Ausgabe:

2
4
6

Pure Lambda: Beispiel 2/2

Vermeide Lambdas, die von außen manipuliert werden!
(wichtig im Bezug auf Multi-Threading sowie Wartbarkeit)



```
void main() {  
    int[] factor = {2}; // Alternativ würde das auch mit ArrayList, LinkedList usw. funktionieren.  
    var stream = Stream.of(1, 2, 3)  
        .map(number -> number * factor[0]);  
    factor[0] = 0; // Not pure! Das Lambda wird von außen verändert!!  
    stream.forEach(IO::println); // Erst hier wird der Stream ausgewertet (Lazy Evaluation).  
}
```

Ausgabe:

0
0
0



Pure Lambda: Closure

In Java bezeichnet ein Closure eine **anonyme Funktion** (aka Lambda), die auf **Variablen aus ihrem umgebenen Kontext zugreifen** kann, selbst wenn dieser **Kontext bereits beendet ist** (aufgrund von Lazy Evaluation).

Closure

Lambda

```
IntConsumer addAndPrint() {  
    int baseValue = 100;  
    IntConsumer result = value -> {  
        int result = baseValue + value;  
        IO.println(result);  
    };  
    return result;  
}
```

// Innerhalb der addAndPrint-Methode wird ein Lambda initialisiert.
// Das Lambda darf auf Variablen außerhalb seines eigenen Geltungsbereichs zugreifen.

Closure-Variablen, die innerhalb eines **Lambdas** verwendet werden, müssen entweder **final** oder **"effectively final"** sein.

```
void main() {  
    addAndPrint().accept(50);  
}
```

// Output: 150

↑
*Evaluation außerhalb von addAndPrint().
Lambda greift trotzdem auf baseValue zu.*



Pure Lambda: Effectively Final

In Java bezeichnet der Begriff **effectively final** eine **lokale Variable**, die nach ihrer Initialisierung **nicht mehr verändert (zugewiesen) wird**, auch wenn sie **nicht explizit mit dem final-Schlüsselwort deklariert wurde**.

```
class EffectivelyFinal {  
  
    void main() {  
        int baseValue = 100; // effectively final variable  
  
        IntConsumer addAndPrint = value -> {  
            int result = baseValue + value;  
            IO.println(result);  
        };  
  
        addAndPrint.accept(50); // Output: 150  
    }  
}
```

```
class EffectivelyNotFinal {  
  
    void main() {  
        int baseValue = 100;  
  
        Consumer<Integer> addAndPrint = value -> {  
            int result = baseValue + value;  
            IO.println(result);  
        };  
  
        baseValue = 42; // Cannot compile. Variable is no longer effectively final  
  
        addAndPrint.accept(50); // Output: none  
    }  
}
```



FYI: Die Variable darf sich nicht ändern, d. h., es ist **keine Neuzuweisung erlaubt**. Was erlaubt ist, den Objektzustand der Variablen zu ändern. Daher war das vorherige "Pure Lambda Beispiel 2/2" kompilierbar:

`int[] factor = {2}; ... lambda; ... factor[0] = 0;` *// effectively final, weil Zustandsänderung, aber keine Neuzuweisung. Nicht pure.*



Ausblick: Gatherer (Stream API)

Die Stream-API bietet einen umfangreichen, wenn auch festen Satz von intermediate Operationen wie map, filter, reduce, und sorted.

Diese Operationen sind für viele Aufgaben geeignet, aber der feste Satz von Zwischenoperationen bedeutet, dass sich einige komplexe Aufgaben nicht einfach als Stream ausdrücken lassen...

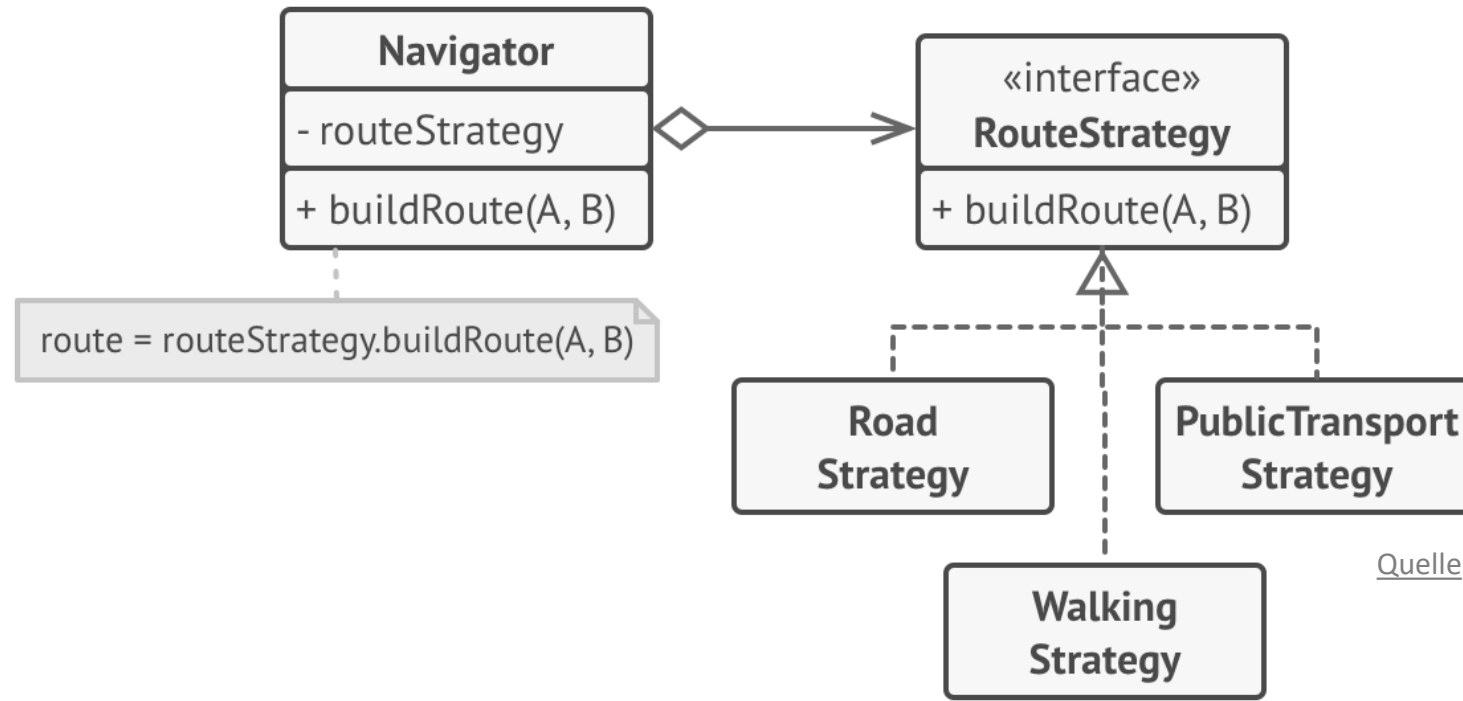
Beispiele: Sliding Windows, fold, scan

Gatherer erlauben es, **eigene intermediate Methoden** für Streams zu schreiben. 🧙‍♂️

Siehe: <https://openjdk.org/jeps/461> (ab JDK 24).

Exkurs: Strategy Pattern obsolete?

Das [Strategy-Pattern](#) erlaubt einer Klasse (z. B. Navigator), die etwas Bestimmtes auf viele verschiedene Arten macht (z. B. RouteStrategy), in separate Klassen (Strategien) auszulagern (z. B. RoadStrategy, WalkingStrategy, PublicTransportStrategy).



Früher musste für jede Strategie eine separate Klasse erstellt werden, die das entsprechende Interface implementiert. Heute können stattdessen einfach Lambdas übergeben werden.

Lambda – Warum?

Lambdas lassen sich flexibel „on-the-fly“ erzeugen, um eine Interface-Implementierung **ad-hoc oder temporär zu erzeugen**.

Siehe Streams...